

Supplementary material

VCF-RDFizer: source-checkable semantic conversion for linked, policy-aware genomic querying

Elias Crum, Bart Buelens, Ruben Taelman, Gökhan Ertaylan

This supplement preserves the experiment-level evidence behind the three questions in the main text. Sections S1–S2 identify the evaluated builds and inputs; Sections S3–S4 specify the representation and checks; Section S5 gives the linked-query protocol; and Sections S6–S8 retain measurements, configuration coverage and defects. Successful tests, negative tests, incomplete checks and unresolved oracle failures are distinguished.

Contents

S1 Reproducibility and execution	2
S1.1 Experiment-to-release ledger and unresolved archive fields	3
S2 Input and version coverage	4
S3 Representation, related tools and implementation	5
S3.1 Documentation-based comparison and capability scope	6
S3.2 Conversion and incremental representation construction	6
S4 Validation specification, scope and sensitivity	11
S4.1 Queries and normalization	11
S4.2 Scope of the campaigns	12
S4.3 Oracle independence and interpretation limits	14
S5 Linked-query protocol, policies and detailed results	14
S5.1 Linksets and release evaluation	14
S5.2 Case definition and conventional comparator	15
S5.3 Outputs, stage costs and applied changes	15
S6 Conversion, storage and retrieval measurements	17
S6.1 Record and sample ladders; artifact accounting	17
S6.2 Costs with different meanings	18
S6.3 Retrieval paths and conventional baselines	18
S6.4 Setup amortization is workload-specific	20
S6.5 Record-scaling figure and complete large-graph outcomes	20
S7 Configuration coverage, operating modes and memory feasibility	21
S8 Defects, corrections and remaining gaps	22

S1 Reproducibility and execution

Release and image. The tagged benchmark configuration uses VCF-RDFizer v3.1.0 (commit d3b34d5) from `ecrum19/vcf-rdfizer:v3.1.0`, with resolved image digest

`sha256:1904e96dde12ab2e2e70d8ee1267765c293ab100a8bd1b14d5b009b2bf8e34aa`

Image rebuilding is disabled during execution so that all cells in the tagged run use the same container. The per-cell manifests retain the actual commit and image identifiers used for each measurement. v3.1.0 bundles the VCF Core SHACL shapes from vocabulary commit 5bfd19d. The regional-retrieval cells postdate v3.1.0 (they need the regional runner and `tabix`) and were produced by commit 3da1a5a. The use case (Section 3.2 of the main text) converts with the v3.2.0 image and links and governs with pre-release builds of the linkers and policy plug-in, recorded per cell: uncommitted `feature/spdi-linker` trees on v3.2.0 (f266db5) for arms 1 and 2, each identified per cell by a recorded diff or tree hash, and commit 578c2be for arm 3. v3.3.0 (commit 7ab2300, image sha256:1838720d...06432) releases them, adding two refusals that none of these runs meets: an empty selector list, and an RDF input with no VCF triples.

Pinned toolchain. VCF-RDFizer v3.1.0 fixes its dependencies inside the container image: RMLStreamer 2.5.0, hdtc 1.1.0, pycottas 1.1.0, DuckDB 1.5.5, cyvcf2 0.34.0, pyshacl 0.30.1, Comunica 5.3.0 for N-Triples, @comunica/query-sparql-hdt 5.0.1, the COTTAS Comunica engine 0.1.0, bcftools from the base distribution, and QLever pinned by image digest. `rdf2hdt` and `hdt2rdf` are compiled from the hdt-cpp repository at image build time, so their exact revision is fixed by the image digest rather than by a version number; the resolved image for each benchmark run is recorded with its command and metrics.

Hosts. Experiments run on two physically separate, identically specified hosts (`vcf-bench-1` and `vcf-bench-2`): Ubuntu 24.04.4 LTS, kernel 6.8.0-85-generic, Intel Xeon Silver 4309Y at 2.80 GHz (8 cores), 33.6 GB RAM with swap disabled, a 210 GB SSD, Docker 29.7.2, and Python 3.12.3. Each complete experiment is assigned to one host and executed without concurrent benchmark jobs, keeping comparisons within-host. On a shared calibration workload the two hosts produced the same graph: identical triple sets, a byte-identical COTTAS artifact and HDT index, and N-Triples and HDT files that differ only in line order and in the HDT header’s `dcterms:issued` timestamp. Their speed differed: the conversion phase took 26s on `vcf-bench-1` and 37s on `vcf-bench-2`, and validation 156s and 208s. Timings are therefore compared only within a host; the calibration characterizes, rather than eliminates, variation between hosts. Retrieval at scale (Table S13) ran outside the campaign, on a third host, `vcf-bench-3`: the same CPU, 8 cores and 31.3 GiB of RAM with swap disabled, under Ubuntu 22.04.5 and kernel 5.15. It queried graphs built by the published v3.1.0 image, with the validation runner of commits 245fe1f (N-Triples) and 5ed75c8 (HDT and COTTAS), which select the thirteen core questions.

What each cell records. The harness creates a fresh output directory for each cell and records its command, host, tool commit, container image, exit status, and wall time. Tool reports provide stage timings, peak resident set size, triple counts, artifact sizes, and validation outcomes; workspace monitoring records transient disk use separately from final artifact size. Input sizes used in performance comparisons are uncompressed VCF bytes, while the corpus table reports compressed download sizes. Replicated comparisons report variability or confidence intervals; single-run corpus measurements describe the tested input and configuration without estimating run-to-run variation. The per-cell records, normalized query results and comparison reports are recorded in the testing repository; a persistent archive of the complete evaluated snapshots is not yet deposited.

The campaign. The campaign comprises 143 recorded cells over 96.2 h of measured wall time (Table S1). Every executed conversion cell records its own argv, host, tool commit and image digest, the two mutation-score cells their tool commit, and the integrated record of all cells is archived with the testing repository. A non-zero exit is classified by what the cell asserted: two

cells were required to be refused and were (*refused*), two recorded an exit status without asserting an outcome (*recorded*), and three were skipped by a stated guard. No asserted expectation failed. This does not mean that all cells completed successfully: the status categories and denominator are retained below.

Table S1: The benchmark campaign, grouped by the question each experiment answers. *Recorded*: a non-zero exit that the cell observed without asserting an outcome; *refused*: the cell required the configuration to be rejected, and it was; *skipped*: a guard refusal with a stated reason. Hosts: **vcf-bench-1** (b1), **vcf-bench-2** (b2). Every conversion cell was produced by the published v3.1.0 image; the two mutation-score cells ran the same commit’s test harness on the host. Section numbers prefixed S are in this document; the others are in the main text.

Experiment	Cells	Outcome	Host	Hours	Question answered
<i>Conversion, scaling and storage</i>					
Records scaling	10	all OK	b1	10.99	How cost grows with records (§3.3)
Sample representation	30	29 OK, 1 skipped	b2	19.69	How the two profiles grow with samples (§3.3)
Storage mode	8	all OK	b1	12.87	Peak workspace of plain vs space-optimized (§3.3)
Corpus breadth	10	9 OK, 1 skipped	b2	26.90	HDT and COTTAS on ten public inputs (§3.3)
<i>Input and configuration coverage</i>					
Representation axes	20	all OK	b1	4.82	VCF 4.1–4.5; cost of INFO and header representations (§S2, §S7)
Difficult inputs	16	13 OK, 2 recorded, 1 skipped	b1	0.10	Eleven awkward VCFs and the demonstration linkers (§S2)
Covering set	10	all OK	b1	0.35	Every option value and pair (§S7)
Modes	7	all OK	b1	0.03	Each operating mode on its own (§S7)
Feasibility	9	all OK	b1	9.03	Memory ceiling × configuration (§S7)
<i>Validation and retrieval</i>					
Robustness	9	all OK	b1	0.25	Determinism, round trip, mutation score (§3.1, §S4)
Equivalence	8	6 OK, 2 refused	b2	6.84	Four engines, both profiles, three artifacts (§S7)
Query cost	6	all OK	b1	4.27	SPARQL vs cyvcf2 on identical work (§3.3)
Total	143	136 OK, 2 recorded, 2 refused, 3 skipped		96.16	

Distribution and maintenance. Each tagged VCF-RDFizer version is published across PyPI, Conda-Forge, and Docker Hub, so users can choose pip-, conda-, or container-based installation with matched versions, under the MIT License [1]. The GitHub README is the primary user guide: quick-start commands, end-to-end examples, command-line option documentation, and troubleshooting notes. The project follows a release-oriented workflow with versioned release notes, public issue triage, and a unit-test suite that runs on Linux, macOS and Windows.

S1.1 Experiment-to-release ledger and unresolved archive fields

The experiment families in Table S2 must not be pooled into a single success rate or attributed to a single release. In particular, the 657.4M-triple conversion/retrieval graph and the 662.4M-triple normalized use-case graph are different artifacts.

[AUTHOR QUERY: Archive the uncommitted arms 1–2 source trees and record the complete v3.3.0 image digest.]

Table S2: Build and assessment boundaries. A commit or image identifies executable state only if all local changes and external resources are also retained.

Family	Recorded state	Assessment boundary
143-cell base campaign	v3.1.0, d3b34d5; full image digest above	Conversion, artifacts, fixtures, 984 applicable paired comparisons and configuration tests.
Regional retrieval	3da1a5a	Later runner with tabix; 11,160 regional executions.
Large-graph retrieval	v3.1.0 graphs; runners 245fe1f, 5ed75c8	Separate host; thirteen summary queries without SHACL.
Linked use case	v3.2.0 conversion; uncommitted pre-release trees on f266db5 in arms 1–2; 578c2be in arm 3	SPDI, gene links and policy views. v3.3.0 7ab2300 is the subsequent release, not the exact state of every run.
Review follow-ups	Default-profile mutation rerun; 250,000-record NG131FQA1I check	v3.1.0 (d3b34d5, image sha256:1904e96d...34aa) on vcf-bench-2; QLever, expanded profile; the NG131FQA1I rerun without shapes; v3.1.0’s size gates.

The public vocabulary citation [2] describes a separately versioned release. The shapes bundled with the base experiment are specifically commit 5bfd19d; the two should not be silently treated as identical.

S2 Input and version coverage

The public corpus. Table S3 lists the ten public VCFs. Download commands and canonical filename definitions are maintained in the testing repository.

Table S3: Public VCF corpus. File names are canonical .vcf.gz stems; later tables abbreviate the GIAB identifiers. Assembly and sample counts were verified from each file’s header. Sizes are approximate compressed download sizes; the tool processes the uncompressed stream.

File	Provider	Assembly	Samples	MB (gz)	Evaluation role
NG1N86S6FC	Sequencing.com	GRCh38	1	379	Consumer WGS
NG131FQA1I	Dante Labs	GRCh38	1	224	Consumer WGS
NB72462M	Nebula Genomics	GRCh38	1	341	Consumer WGS
60820188475559	PGP upload	GRCh37	1	325	SNP-only profile
60820188474283	Dante Labs	GRCh37	1	222	Consumer WGS
OG00R_HG002	precisionFDA	GRCh38	1	69	GIAB Truth Challenge v2
1000G_phase3_chr20	1000 Genomes	GRCh37	2,504	327	Phased cohort
HGSVC2	HGSVC2	GRCh38	32	31.5	Multi-sample SVs
HG004_GRCh38	GIAB v4.2.1	GRCh38	1	149	Benchmark genome, chr1–22
HG005_GRCh38	GIAB v4.2.1	GRCh38	1	139	Benchmark genome, chr1–22

Declared VCF versions. Five single-purpose fixtures covering VCFv4.1 through VCFv4.5, plus one file with no ##fileformat declaration, all converted with exit code 0. The generated RML mapping for each version is byte-identical apart from the asserted file class (vcfc:VCF41File through vcfc:VCF45File), which locates version awareness in the typed file resource rather than in field-level rules. Field handling is therefore data-driven: the VCFv4.5 fixture’s local-allele FORMAT keys (LAA, LAD) and the VCFv4.4 fixture’s phase-set key (PS) are carried through as ordinary FORMAT entries, raising the per-record triple yield from 210 (4.4) to 252.5 (4.5) on otherwise matched two-record fixtures. This is preservation, not interpretation: nothing in the graph resolves a local allele index to a global allele, so VCFv4.4/4.5 support is lexical carry-through

pending a dedicated local-allele semantics layer. The undeclared-version file also converted, via the documented fallback.

Real cohorts. Nine of ten corpus inputs converted end to end (the first 250,000 records of eight files with headers retained, and HG005_GRCh38 whole), with HDT and COTTAS built and decode-checked for each. The tenth, the 2,504-sample 1000 Genomes chromosome-20 call set, was skipped by the benchmark harness rather than refused by the tool: the harness withholds any corpus file with more than 1,000 sample columns from the expanded arm, because the expanded representation emits one call resource per record and sample. Its full sample dimension was exercised instead through 10,000-record slices carrying all 2,504 samples, which converted in both representations; Section S6 quantifies why the expanded arm cannot be run on the whole file at this disk budget. The tool also applies a workspace check: before converting, it estimates the expanded workspace from the record and sample counts and refuses the run when that estimate exceeds three quarters of the free space, naming the condensed representation as the alternative.

Difficult inputs. Eleven adversarial fixtures were run (Table S4; fixture files are named `awkward_*` in the testing repository). Nine converted and passed validation, including CRLF line endings, symbolic alleles, breakend ALT syntax, ploidy 4 and 3 in the same file, half-calls and haploid calls, missing values in every optional field, header keys declared but never used, the three-way `PASS`/`.`/`failed`-filter distinction, and a header-only file with no data records. Two cells exited non-zero. One is a refusal to convert: the truncated gzip stream, rejected in 0.58 s at the decompression stage with a clear diagnostic, which is the intended behavior. The sites-only file produced 245 triples and an HDT artifact, but its paired validation was not completed (Section S8).

Table S4: Difficult-input fixtures. Outcomes are reported separately from validation. Neither a successful conversion nor an intended refusal establishes fidelity; the sites-only check is incomplete. Triple counts are for the complete graph the run produced.

Fixture	Outcome	Triples	Validation	Note
CRLF line endings	converted	262	PASS	No stray <code>\r</code> in the last field's literal
Symbolic alleles	converted	453	PASS	<code></code> not treated as a base string
Breakend ALT	converted	417	PASS	Breakend ALT syntax preserved
Mixed ploidy	converted	350	PASS	Ploidy 4 then 3 followed the data
Half and haploid calls	converted	446	PASS	Four genotype cases stayed distinct
Missing values	converted	321	PASS	Missing-field fixture; missingness follows the field-specific representation
FILTER variants	converted	486	PASS	<code>PASS</code> , <code>.</code> and a failed code are three states
Declared FORMAT unused	converted	293	PASS	Header declaration represented although never used
Zero data records	converted	189	PASS	Header-only input; no data records
Sites only	converted; validation not completed	245	incomplete	Graph and HDT artifact produced
Truncated gzip	refused at decompression	—	not reached	Refused in 0.58 s with a decompression diagnostic; intended

The successful sites-only frequency graph used in the linked case (Section S5) does not repair the malformed validation fixture. It establishes that the frequency-join workflow ran, not that a conformant sites-only input completed the full paired validation suite.

S3 Representation, related tools and implementation

VCF-RDFizer emits RDF in the VCF Core Vocabulary (`vcfc:`, <https://w3id.org/vcf-core/vocab#>) [2], a separately maintained vocabulary that models the VCF specification itself, version by version, for VCFv4.1 through VCFv4.5. Its terms align with established resources for genomic coordinates (FALDO [3]), variant typing (the Sequence Ontology [4]), genotype and sample con-

structs (HERO [5]), and features of the GA4GH VRS schema v2.0.1 [6], so VCF-specific structure can be connected to existing terms. The model preserves three information layers: file and header assertions, variant-record assertions, and sample and genotype assertions (Figure S1). The vocabulary carries classes, properties, domains and ranges but no OWL restrictions; value and cardinality constraints are SHACL shapes, which the validation layer applies (Section 2.2).

Sample-level data can be emitted in two profiles. In the *expanded* profile every sample value is its own resource, which makes genotypes directly queryable. In the *condensed* profile values are stored as sample-ordered vectors, reducing the number of RDF resources while requiring decoding for access to individual sample values. Both profiles keep the same file, header and record context. Let V be the number of variant records, S the number of samples, and F the number of FORMAT keys per record. For a uniform FORMAT layout, the expanded profile adds VS sample-call resources and VSF FORMAT-value resources, whereas the condensed profile adds V cohort matrices and VF value vectors. Excluding the structure common to both, and assuming bounded ploidy and FORMAT-value arity, the sample-layer triple counts scale as

$$\begin{aligned} T_{\text{expanded}} &= \mathcal{O}(S + VSF), \\ T_{\text{condensed}} &= \mathcal{O}(S + VF). \end{aligned} \tag{1}$$

The condensed profile removes the per-sample factor from the repeated RDF structure, but still stores all VSF values inside vector literals, so its size in bytes keeps growing with S even though its triple count does not (Section 3.3).

Core entities are minted from deterministic IRI templates over the source file and record position, so identifiers are reproducible but file-scoped: observations of the same variant in two files receive different IRIs. Identity across files is added by linking (Section 2.2); GA4GH VRS computed identifiers [6] could serve beside NCBI SPDI and are future work.

S3.1 Documentation-based comparison and capability scope

The comparison records the inspected documentation rather than a shared-input benchmark. An absent mark means a capability was not found in that documentation, not that the software cannot provide it. Package channels and mapping extensibility describe practical interfaces; they are not direct measures of semantic fidelity or usability.

Related work. Earlier converters include the *rdf-vcf* reader and the web-based *VCF2RDF*, an isomorphic mapping to RDF/JSON-LD [7, 8]; a 2013 BioHackathon *vcf2rdf* using FALDO and tool-specific terms [9]; and JVarKit’s *VcfToRdf*, TOGOvar’s Rust *VCF2RDF* and SPARQLing Genomics’ *vcf2rdf* [10, 11, 12, 13]. General tools also reach RDF: BioInterchange emits GFVO from GFF3, GVF and VCF 4.1–4.2 [14], and *bio-vcf* serializes through templates [15]. Pipelines such as VariantKG build on these converters [16]; AgrolD names a VCF2RDF component whose code was not located in the documentation assessment [17]; and Semantic Beacons exposes Beacon API responses as RDF through RML without materializing a file [18]. Table S5 compares what these systems document; measuring conversion cost and fidelity against them on a shared input remains to be done.

S3.2 Conversion and incremental representation construction

Conversion has five stages: (1) each VCF is split into record, header-line and file-metadata TSV logical sources; (2) modular RML rule sets [19], authored in Turtle and executed by RMLStreamer [20], map them to RDF; (3) emitters add the remaining fields and the samples, and every part is appended to one N-Triples aggregate; (4) optional gzip or Brotli packages and queryable HDT and COTTAS representations are built from the aggregate; and (5) every representation is validated, packaged and reported. Options keep these concerns separate (`-rdf-storage-mode`,

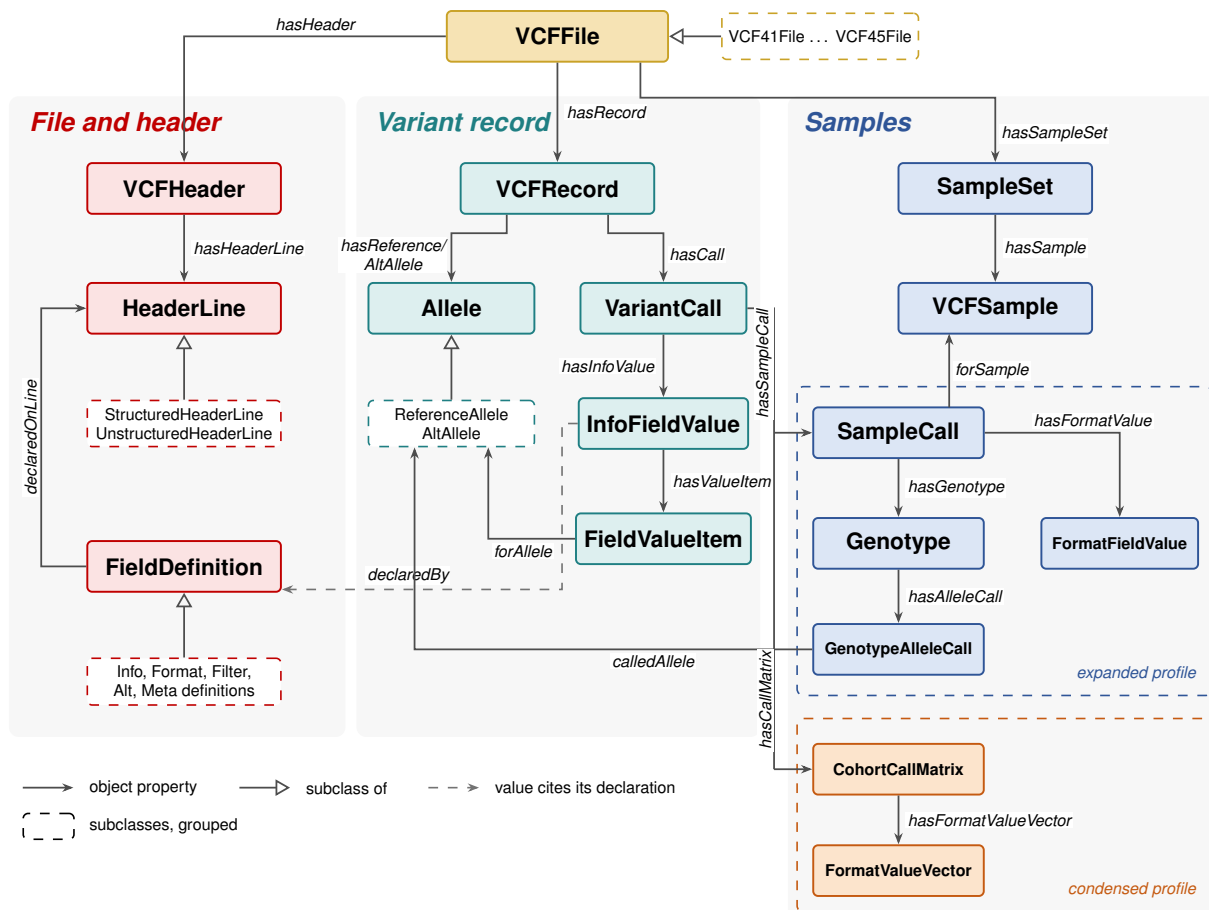


Figure S1: **VCF Core Vocabulary, simplified class diagram.** The backbone of the model VCF-RDFizer v3.1.0 emits, drawn from the ontology bundle it ships; all class and property names carry the *vcfc:* prefix, <https://w3id.org/vcf-core/vocab#>. A **VCFFile**, typed with its version-specific subclass, links to three layers. In the **header** layer, header lines carry the INFO, FORMAT, FILTER, ALT and META definitions that values cite through **declaredBy**. In the **record** layer, each **VCFRecord** has ordered alleles and a **VariantCall** carrying the INFO values, whose structured items retain their record, field and allele context. In the **sample** layer, the call links either to per-sample **SampleCalls**, genotypes and allele calls (*expanded profile*) or to one **CohortCallMatrix** of sample-ordered vectors (*condensed profile*). Structural-variant, phasing, local-allele and value-typing classes are omitted; the full vocabulary, with worked examples, is documented at <https://ecrum19.github.io/vcf-core-vocabulary/>.

Table S5: VCF-to-RDF converters and related semantic genomic-variation workflows, compared on the capabilities this study targets. Assessments are based on each system’s publicly available documentation. This is a feature comparison, not a measured one: no other converter was run on the same input here, so the table shows what each system documents, not that one converts better or faster.

Tool / workflow	CLI	RDF model	Reprod. workflow	Metadata + genotype	Extensible mappings	RDF compr.	Public release
<i>Dedicated VCF-to-RDF converters</i>							
VCF-RDFizer (this study)	✓	✓	✓	✓	(✓) ^c	✓	✓
TogoVar VCF2RDF [11]	✓	(✓)	✓	(✓)	(✓)	–	✓
JVarkit VcfToRdf [10]	✓	(✓)	✓	(✓)	–	–	✓
SPARQLing Genomics [12]	✓	(✓)	✓	✓	(✓)	–	(✓)
VCF2RDF (web) [8]	(✓)	✓	(✓)	(✓)	–	–	(✓)
BioHackathon vcf2rdf [9]	✓	(✓)	(✓)	(✓)	–	–	(✓)
<i>General genomic-format tools</i>							
BioInterchange [14]	✓	✓	✓	(✓)	–	–	✓
bio-vcf [15]	✓	–	✓	(✓)	✓	–	✓
ruby-rdf/rdf-vcf [7]	✓	–	(✓)	(✓)	–	–	✓
<i>Knowledge-graph construction pipelines</i>							
VariantKG ^a [16]	(✓)	(✓)	(✓)	✓	–	–	(✓)
AgroLD VCF2RDF ^b [17]	–	(✓)	–	–	–	–	(✓)
<i>Virtual semantic access</i>							
Semantic Beacons [18]	–	(✓)	(✓)	(✓)	✓	–	(✓)

✓ supported and documented; (✓) partial or limited; – not found in public documentation. **CLI**: command-line use in pipelines. **RDF model**: documented vocabulary for the output. **Reprod. workflow**: documented, reproducible VCF-to-RDF route. **Metadata + genotype**: header metadata kept with genotype and sample data. **Extensible mappings**: rules editable without changing the converter. **RDF compr.:** integrated compressed RDF output. **Public release**: distributed package, executable or container.

^aConverts VCF through SPARQLing Genomics’ vcf2rdf after SnpEff annotation, then adds its own ontology for CADD scores and patient metadata. ^bNamed in [17] as part of the AgroLD ETL; the assessment reflects public documentation inspected in September 2026. ^cRML maps the record backbone, about 5% of the triples; the content-bearing layers (samples, INFO, QUAL, header) come from the wrapper’s emitters, configured by options rather than mappings (Section 2.2).

`-rdf-compression`, `-representations`, `-artifact-compression`), so an `.hdt` or `.cottas` file can be kept for querying while a `.gz` or `.br` package is made for transport.

RML maps the record backbone: nine triples per record, 5.3% of the HG005 graph’s triples. The content-bearing layers — samples, structured INFO, QUAL and the header, 94.7% of that graph — come from the tool’s emitters, configured by options, because the implemented mapping path delegates per-value datatype selection to emitters (a missing QUAL, for instance, is written `".^^vcfc:Null`). The TSV step exists because VCF is not a native RML logical source.

In **plain** storage mode the parts are appended to one uncompressed `.nt` graph; in **space-optimized** mode each part is gzip-compressed and appended as a member of one concatenated-gzip `.nt.gz` stream, which standard gzip readers expose as one N-Triples sequence. Either way each part is deleted once appended, so all parts and a full uncompressed aggregate never coexist.

HDT and COTTAS are built one record-safe chunk at a time, so at most one uncompressed chunk exists while they are built. HDT parts are produced with `rdf2hdt` [21, 22], merged pairwise with `hdtc` [23, 24] and indexed. COTTAS parts are produced with `pycottas` [25] and merged in one streaming *k*-way pass with an `spo` index (v3.2.0 adds the other orders); COTTAS files are queried by a Comunica engine that evaluates SPARQL over the Parquet table with DuckDB [26].

Table S6: VCF-RDFizer capability status and benchmark scope. “Supported” describes the release; the evaluation column states what this study measured, so that support is not mistaken for evidence. Section numbers without the S prefix are in the main text.

VCF component / capability	Release	Evaluation	Notes / limitations
Core record fields (CHROM, POS, ID, REF, ALT, FILTER)	Supported (v3.1.0)	Validated	Q1–Q4 equal in completed comparisons; the broader suite has the follow-up failures reported in the main text (Section 3.1)
QUAL	Supported	Q11 record digest	Typed literal for every record; a missing QUAL is <code>vcfc:Null</code>
INFO	Supported; structured by default	Q11 and Q12 digests	Typed value per record and key, per-allele items for Number=A/R/G, plus the complete <code>infoRaw</code> string
FORMAT and sample/genotype values	Supported	Q5–Q6 exact; Q13 digest	Half-calls, haploid calls and ploidy 4/3 survive conversion (Table S4); called allele, sample index and phasing require the deeper shapes for the tested mutations
HEADER metadata	Supported	Census queries and shapes	Header attribute-value agreement requires the deeper shapes, not merely the default profile
Multi-sample conversion	Supported	Measured, 1–2,504 samples	Condensed adds 5 triples per sample; expanded adds 25 per sample per record (627M triples for 10,000 records at 2,504 samples), so the whole cohort was converted in neither (Section 3.3)
Structural and symbolic alleles	Lexically supported	HGSVC2 converted; Q2 passes on symbolic and breakend fixtures	ALT remains the source lexical field; allele-value mutations are caught only by the full shape profiles
Queryable HDT, COTTAS	Supported	Measured	118 HDT and 34 COTTAS artifacts, all decode checks passed; COTTAS smallest throughout (Section 3.3)
Gzip/Brotli packaging	Supported	Covering set (Section S7)	Packages must be unpacked before querying; no codec comparison
Policy annotations	Plug-in v0.2.0 with v3.3.0	Real genomes, up to a whole genome	ODRL on files and declared graph selections, DUO purposes; every view checked against a VCF-side oracle; simulated consents, single-sample files; governed release, not anonymization (Section 3.2)
External linking	SPDI, Ensembl-gene and MyVariant linkers with v3.3.0	Real genomes	SPDI joins eligible alleles under the stated reference/normalization assumptions; tier 3 confirmed 92–93% of rsID rewrites (Section S5)

Before the raw RDF is removed, every artifact is decoded with its native reader and its triple count compared with the source’s; a mismatch fails the run. The following subsection details chunking.

Chunked construction of HDT and COTTAS. The representation stage reads the plain or gzip aggregate incrementally, respects newline-terminated N-Triples record boundaries, and materializes one raw chunk in a Docker-managed workspace. Each selected representation builder consumes that chunk before it is deleted and the next one is created (Figure S3). At most one uncompressed RDF chunk exists at a time; the final parts, merge intermediates, indexes and converter scratch space still require storage, but no full decompressed duplicate of the aggregate is created. HDT chunks are converted with `rd2hdt` from `hdt-cpp` [21, 22] and merged pairwise in a balanced tree with `hdtc create` [23], which performs the same logical merge as HDTCat [24] under a fixed memory budget and without Java; an unpaired file is carried into the next round, and each merge’s scratch directory is removed as soon as that merge completes. The final HDT is indexed by `hdtc`, producing the sibling `.hdt.index.v1-1` index without executing a query; intermediate indexes are not retained. COTTAS chunks are converted with `pycottas` [25] and merged in one streaming k -way pass rather than pairwise, so no global in-memory hash table or full-size temporary spill is built. In v3.1.0 the merged output carries a fixed `spo` index; v3.2.0 adds the other index orders. Each COTTAS operation runs in an isolated temporary working directory, so that its local DuckDB scratch database cannot be reused by a later chunk or merge. Chunk conversion, merging, final indexing, validation, and packaging are timed separately.

Chunk sizes. By default a chunk targets 512 MiB of uncompressed N-Triples, bounded below

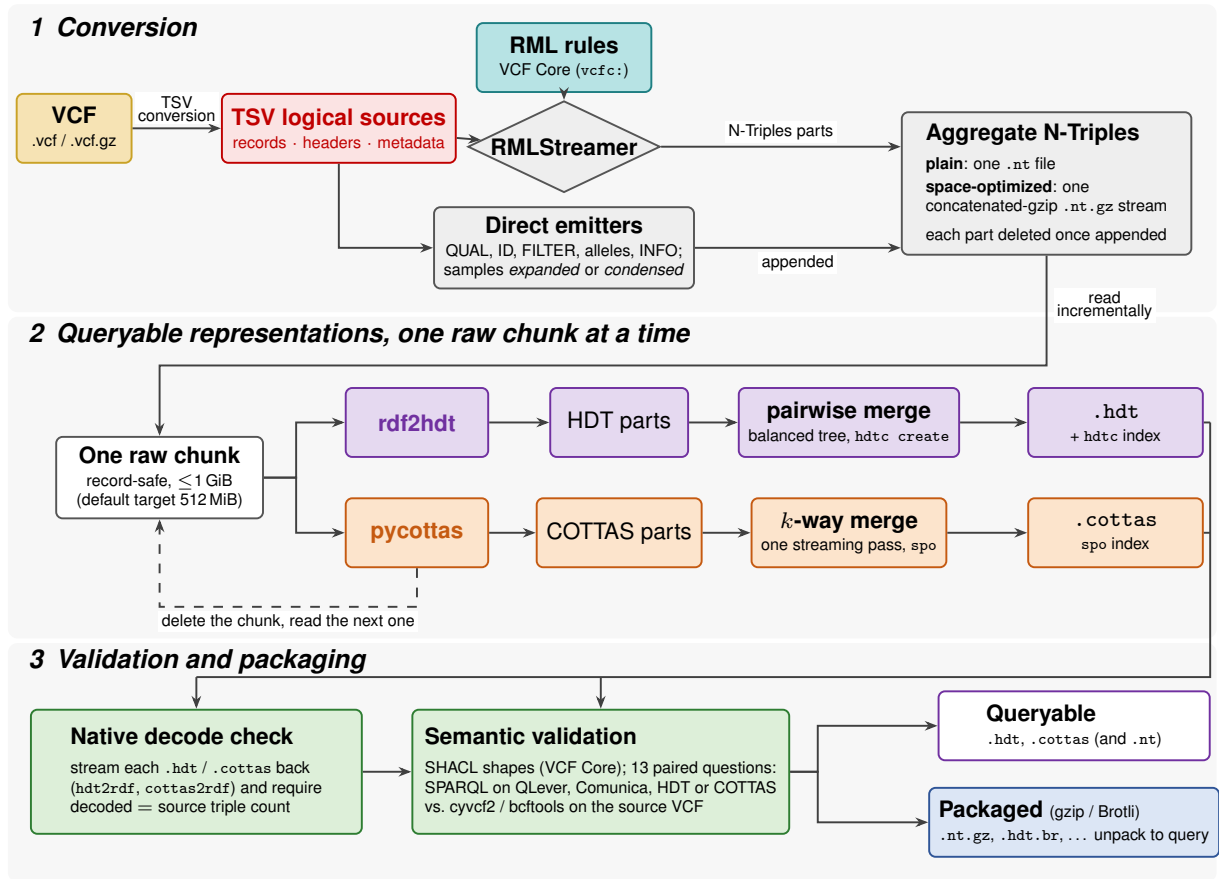


Figure S2: **VCF-RDFizer v3.1.0 pipeline**. (1) **Conversion**: the VCF is split into three TSV logical sources; RMLStreamer applies the RML rules to emit VCF Core N-Triples, and direct emitters add the row-dependent fields whose datatype is chosen per value (QUAL, ID, FILTER, alleles, structured INFO) and the samples in the *expanded* or *condensed* profile. Every part is appended to one aggregate and then deleted: a plain .nt file, or in space-optimized mode a concatenated-gzip .nt.gz stream. (2) **Queryable representations**: the aggregate is read incrementally and only one record-safe raw chunk exists at a time; **rdf2hdt** and **pycottas** each consume it before it is deleted. HDT parts are merged pairwise in a balanced tree with **hdtc** and indexed; COTTAS parts are merged in one streaming *k*-way pass with an **spo** index. (3) **Validation and packaging**: each artifact is decoded back and its triple count compared with the source; the graph is then checked against the VCF Core SHACL shapes and against thirteen questions answered both by SPARQL and by a VCF parser. HDT and COTTAS files are queryable as written; gzip or Brotli packages are for transport and must be unpacked before querying.

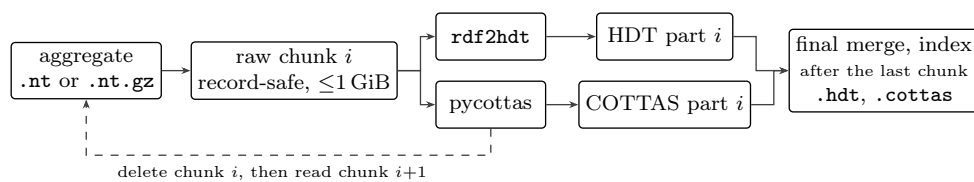


Figure S3: One chunk at a time: each record-safe chunk is consumed by every selected builder and deleted before the next is read.

by 128 MiB and above by 1 GiB (`-chunk-target-bytes`, `-chunk-min-bytes`, `-chunk-max-bytes`). For constrained workspaces a lower-peak setting is 128 MiB target, 64 MiB minimum and 256 MiB maximum, trading additional chunks and merges for a lower temporary peak. Every experiment used the defaults except six cells of the memory-feasibility sweep, which used the lower-peak setting (Section S7).

S4 Validation specification, scope and sensitivity

S4.1 Queries and normalization

Readable output need not be faithful output. The validation layer (`-validate`) therefore asks the graph thirteen standard VCF inspection and quality-control questions [27, 28, 29] in SPARQL, and derives each expected answer from the source VCF with `cyvcf2`, using `bcftools query` for exact FILTER text (Table S7). Fourteen graph-integrity preflight queries run first: blank nodes, empty values, untyped missing tokens, the POS datatype, the cardinality of core record properties, the sample representation, the sample and genotype inventory, and the triple count.

Table S7: The thirteen-query equivalence workload. Each SPARQL result is normalized and must equal a result derived from the source VCF by the separate parser-based path (Q4’s exact FILTER text via `bcftools query`). Ti/Tv, call rate and allele frequency are derived only after their underlying counts agree. Q11–Q13 count values in up to 256 buckets keyed on the first byte of SHA-256 over the IRI and values, which detects value changes that preserve ordinary counts but is not a collision-free proof of equality. For Q9, Q10, Q12 and Q13 the expected result follows the selected INFO and sample representations.

Query	Result compared, fields and scope
<i>Record level — every input</i>	
Q1 Record density	Records per contig and 1-Mb window (CHROM , POS); checks coordinates and integer positions
Q2 Allele shapes	Records per class — SNV, multiallelic, insertion, deletion, substitution, symbolic/breakend, no ALT (REF , ALT)
Q3 Ti/Tv	Transitions, transversions and biallelic SNVs; single-base A/C/G/T alleles only
Q4 FILTER values	Records per exact FILTER string: PASS , . (not applied) or failed codes, combined codes kept whole
<i>Genotype level — inputs with sample columns and GT</i>	
Q5 Genotypes	Calls per sample and genotype class, including missing, haploid and other-ploidy calls; phased and unphased alike
Q6 Allele counts	Sites per recomputed (AC, AN) pair; single-ALT sites with complete 0/1 haploid or diploid calls
<i>Metadata and graph structure</i>	
Q7 File metadata	Exact fileformat , reference and source strings, with markers for absent declarations
Q8 Header census	## lines per header key (INFO , FORMAT , FILTER , contig , ...)
Q9 Predicate census	Triples per predicate against the counts expected from the VCF
Q10 Class census	Resources per rdf:type against the expected inventory
<i>Value preservation — IRI-bound digests</i>	
Q11 Record digest	Records per bucket over each record’s IRI, CHROM , POS , ID , REF , ALT , QUAL , FILTER and raw INFO
Q12 INFO digest	Structured INFO values per bucket, bound to record and key; empty in raw-only INFO mode
Q13 FORMAT digest	FORMAT values (expanded) or sample-ordered vectors (condensed) per bucket, beyond GT

Results must be exactly equal after normalization: strings for Q7 and integer counts elsewhere, including the digest buckets of Q11–Q13, which extend coverage to **QUAL**, **INFO** and **FORMAT**

without guaranteeing value-by-value equality. The comparator keeps contig and sample identifiers, distinguishes PASS from ., and reports missing, extra and differing rows. Q5–Q6 apply only when the source has a GT field, and each input is validated only against its own graph, so inputs on different assemblies are never compared.

The same SPARQL text runs under four engines: QLever [30], the default; Comunica over N-Triples (@comunica/query-sparql-file) [31, 32]; Comunica over HDT (@comunica/query-sparql-hdt) [33]; and the DuckDB-backed COTTAS engine [26]. Any engine can be pointed at any artifact, and `-validation-engine all` runs all four and records whether they agree. The layer also applies the vocabulary’s SHACL shapes in three profiles. The default profile checks cardinality and datatypes; the other two check uniqueness and value agreement, and are costlier (Section 3.1).

What a passing comparison establishes is agreement of the specified summaries: a SPARQL engine and a VCF parser derive the same deterministic summaries from two representations of the same source. It does not establish that the source calls are biologically correct, normalized or concordant with a truth set.

S4.2 Scope of the campaigns

The base campaign attempted validation 78 times and completed 76. It yielded 984 applicable query comparisons with no mismatches and four not-applicable genotype comparisons on the empty-input artifacts. Its paired checks ran on fixtures and slices up to 17.1M triples. The two malformed sites-only validations did not complete and are not successful comparisons.

The whole HG005 graph was decode/count checked in the corpus campaign. Separately, the retrieval experiment compared all thirteen QLever query answers at 657,425,805 triples with the parser, without SHACL (Table S13). Those comparisons are not included in the 984 denominator. The 58.2M-triple consumer-genome follow-up is another separate run; it has three summary mismatches and must not be counted as a validation pass.

The default shape profile ran in 58 base validations, on graphs up to 0.96M triples, with no violations and 0.8–396.8s per run. Repeated conversion produced 17,098,746 triples with identical SHA-256 and no duplicate lines. A packaging round trip reproduced the digest and re-indexing left the artifact unchanged. These assess deterministic output under the tested conditions, not reproducibility of every later pre-release build. **A real genome.** On the first 250,000 records of the NG131FQA1I genome, 58.2M triples and 3.4 times the largest graph validated in the campaign, ten of the thirteen comparisons were exactly equal, but the layer reported a mismatch. Direct diagnostic comparison found every record field equal to the VCF text across all 250,000 records. This narrows the cause of the observed summary failures; it is not a proof of every emitted relationship. The observed causes concern the validation layer and its normalization assumptions. The censuses (Q9, Q10) found 30,910 phase sets, emitted for GATK’s PS field, that the oracle does not yet model; the tool documents the same gap for structural-variant events, gVCF blocks and local alleles. The record digest (Q11) differed because QLever returns a decimal in canonical form: the 1,998 records whose QUAL has trailing zeros hash differently, and recomputing the oracle’s buckets with canonical QUAL reproduces QLever’s exactly. A first attempt with the default shapes exhausted the host’s memory (Section S8), so this run is without them.

Sensitivity. The mutation experiment injected 113 targeted corruptions — wrong allele, wrong coordinate, wrong filter token, wrong file metadata, blank nodes, flipped phasing, corrupted sample index — and the layer with all three shape profiles detected all of them. The two halves contribute differently. The comparison queries alone detect 96 (0.850). The other 17 fall into ten classes the queries structurally cannot see: allele kind and allele value are counted but never compared with REF and ALT, header attribute values and the derived contig count are counted but not read, the joins from a FORMAT value item to its allele and from a field ID to its declaring header line are counted but not checked, and record index, sample index, called allele and phasing

Table S8: Validation evidence, grouped by the question each layer answers. Counts are pass/total, derived from archived reports. Sensitivity is measured on the defined fault set, not estimated for arbitrary faults.

Check	Passed	What it establishes
<i>Is the graph faithful to the source VCF?</i>		
N-Triples syntax (raper)	76/76	The serialization parses
Paired SPARQL vs cyvcf2 (Q1–Q13)	984/984 [†]	Exact equality of normalized query results
Cross-check invariants	1,144/1,144	E.g. transitions + transversions = biallelic SNVs
Cross-engine agreement	25/25	Four-engine validations; 51 other completions used one engine
Native decode and triple count	152/152	118 HDT, 34 COTTAS; source = decoded
<i>Is the pipeline reproducible?</i>		
Determinism, two runs	1/1	Identical SHA-256; 17,098,746 triples, 0 duplicate lines
Compress and decompress	1/1	Lossless through the packaging path
Index idempotence	1/1	Re-indexing does not perturb the artifact
<i>Would a fault actually be noticed?</i>		
Injected faults caught, queries only	96/113 (0.850)	17 faults, spanning 10 mutation classes, are invisible to the queries
with the default shape profile	96/113 (0.850)	Cardinality and datatypes catch none of the 17
with all three shape profiles	113/113 (1.000)	Uniqueness and value agreement catch all 17; both controls still pass
SHACL shapes, default profile	58/58	Cardinality and datatype constraints, 0 violations

[†]Four further comparisons were verified as not applicable and are excluded from the 984 applicable comparisons.

status have no oracle. The full shape set catches every one, and both unmutated controls pass in both profiles, so the shapes are not simply rejecting conformant graphs. But the default profile, which checks cardinality and datatypes, catches none of the 17: with it alone the layer detects 96 of 113, exactly the queries’ score. The two profiles that do catch them check value agreement and uniqueness, self-join the graph, and cost 33.7s and 73.2s respectively on a 2,000-triple graph against 1.7s for the default. v3.1.0 therefore applies the default automatically to inputs up to 512 MiB, and the full set only when requested (`-shacl-profile full`) and only to inputs up to 16 MiB, so the default configuration does not detect these additional faults in the tested set. The guards used different size measures in different modes (Section S8); the full-profile result must not be extrapolated to larger graphs on which those profiles were not run.

Real defects. During development, testing caught three defects that the record counts alone could not show: two in the conversion, caught by the validation layer, and one in the policy plug-in, caught by its own check. A header-only file omitted the samples its header declares, which the predicate and class censuses (Q9, Q10) detected. Expanded genotypes referenced allele resources that a raw-INFO conversion never emitted: a 1,000-record test held 1,972 such references, which the queries counted without following and the shapes reported as 5,916 violations. And in the policy plug-in, a withdrawn file’s header remained in every release view although every record decision was correct; the plug-in’s check caught it, and it is why withholding is defined by ownership. All three are fixed. Section S8 records these and the measurement and reporting defects testing found.

Inputs and encodings. Every declared version from VCFv4.1 to v4.5, and an undeclared one, converted; v4.4 and v4.5 fields such as local alleles are carried, not interpreted. Nine of eleven awkward inputs converted and validated, a truncated gzip was refused with a clear diagnostic, and a sites-only fixture could not be validated because bcftools rejects it (Section S2). Every configuration value validated, shapes included, and all four engines agreed on both profiles, both storage modes and all three artifacts in sixteen validations (Section S7).

S4.3 Oracle independence and interpretation limits

The parser and graph routes read separate representations, but not all assumptions are independent. Predicate and class expectations must encode the converter’s selected profile; allele-count checks share the biological counting definition; and both routes share case definitions and pre-processing in the use case. The missing-allele defect required changes to both the emitter and its census expectation, showing how an incorrect dependency can be shared. Direct VCF-text comparison and structural constraints provide complementary checks, not an infallible oracle.

The Q11–Q13 histograms use finite hash buckets and can miss changes that preserve a histogram. Counts after decoding establish count agreement, not complete RDF-term identity. A passing shape result establishes only its enabled constraints. The full 113-fault detection result applies to the constructed set with all profiles; the default configuration’s observed detection remains 96/113. The default-profile follow-up is completed work, not an outstanding experiment.

S5 Linked-query protocol, policies and detailed results

S5.1 Linksets and release evaluation

Two host-side commands extend a converted graph without changing it.

Linking. `vcf-rdfizer-link` writes a separate N-Triples linkset, so linking never alters the converted graph or its validation. A linker is declared in Turtle at one of three tiers. *Tier 1* rewrites tokens the file already holds, such as rsIDs, into IRIs, without checking them. *Tier 2* joins against a reference pinned by digest and assembly. The `spdi` linker trims each ALT allele to its minimal form and emits its NCBI SPDI expression [34] on the RefSeq accession as an IRI, so eligible, equivalently normalized alleles can share an IRI across contig aliases such as `17` and `chr17`; `ensembl-genes-grch38` links each call to the Ensembl 116 genes its REF span overlaps, with contig aliases. *Tier 3* confirms identifiers against a live service through a short Python resolver: requests are batched under a declared rate and budget, responses are cached with digests, and any failure fails the linker closed, publishing nothing. `rsid-myvariant` confirms rsIDs against MyVariant.info [35]. When the input is RDF, the join fields are read with SPARQL, from an endpoint or from a temporary Oxigraph store.

Policy. `vcf-rdfizer-policy` attaches ODRL policies [36] to graphs and derives one release view per request in three generic steps, each configured in Turtle rather than code. A rule’s target is a resource, or a selection computed by a declared SPARQL selector; region, variant and linked selectors ship with the tool, and others can be declared in a policy file. A profile’s ownership rule states what a withheld resource takes with it: under the VCF Core profile, withholding a record withholds its call, alleles and sample calls, and withholding a file withholds its header too. The implemented ODRL profile then decides, with deny winning and anything no permission covers withheld; purposes come from any RDFS or SKOS vocabulary, by default a subset of the GA4GH Data Use Ontology (DUO) [37]. Selector parameters are injected as an inline **VALUES** block opening the query’s outer group, because SPARQL joins a trailing **VALUES** clause only after the group is evaluated: a region selector written that way selects nothing, and a prohibition on the region fails open. Version 0.2.0 evaluates against a SPARQL endpoint and streams each view from the N-Triples, byte for byte and in input order. Every view is then checked:

- its policy digest matches;
- nothing a binding prohibition owns is present, and nothing lacks a permission;
- no reference dangles, a question put to a second endpoint that serves the view;
- its records equal an oracle that evaluates the same policy over a graph built directly from the VCF text, so an extra record is a leak and a missing one over-withholding.

The plug-in provides governed release, not anonymization: a released genotype remains identifying, views keep the original IRIs, and obligations are recorded but not enforced.

S5.2 Case definition and conventional comparator

To test linking and policy on real data, we answered one clinically recognizable question over real genomes with two downstream implementations that share preprocessing and the case definition: *which participant-variant-gene matches connect an observed allele to a ClinVar classification in the 81-gene ACMG secondary-findings set, SF v3.2 [38], and what may each of three requesters see?* The RDF route converts every input with VCF-RDFizer, links each with the `spdi` and `ensembl-genes-grch38` linkers, evaluates the policies with `vcf-rdfizer-policy` against QLever [30], and asks one SPARQL query per requester over that requester’s view and ClinVar. The conventional route renames contigs to ClinVar’s naming, annotates each genome with ClinVar’s `CLNSIG`, `CLNREVSTAT` and `GENEINFO` using `bcftools` [28], and applies the consents in a Python script. Both read one case definition: the question, the ClinVar review statuses that count (one star or more), and the consents. A gate compares the two match sets for every requester, keyed by participant, gene, contig, position and alleles, and no timing is reported unless they are identical.

The genomic data are real and public; the consents are simulated, and every output says so. Each file carries a consent stated as DUO purposes; a withdrawal is a prohibition, so deny wins. One cohort rule releases records in the list’s 28 cancer-predisposition genes only for clinical care. The requesters are a clinical genetics laboratory (clinical care, CC), a cardiovascular consortium (disease-specific research, DS) and a biobank researcher (general research use, GRU).

Three arms separate integration, breadth and scale. Arm 1 is five genomes from three sources: two Personal Genome Project genomes [39], HG004 and HG005 from Genome in a Bottle [40], and an HG002 precisionFDA submission whose contigs are named 1, 2, ... rather than `chr1`. Arm 2 is 104 unrelated participants of the 1000 Genomes high-coverage call set [41], four from each of its 26 populations, with seeded simulated consents, four of them withdrawn. Arms 1 and 2 are restricted to the 81 genes’ spans (7.1 Mb, Ensembl 116) and normalized with `bcftools` (left-aligned, multi-allelic records split). Arm 2 drops the panel’s INFO fields, which describe all 3,202 panel samples rather than the participant and made up 93% of each participant’s triples; the frequencies the question uses are recorded once, in a sites-only file. Arm 3 is HG005’s whole genome through the same pipeline, so its answer must equal its arm-1 answer. ClinVar’s 2026-09-13 release [42] is restricted to the same spans.

Effort protocol. On arm 1 we counted the lines each route’s author writes, and the lines four changes add and remove: a withdrawal, a requester with a new purpose, a cardiac panel in place of the cancer genes, and a rule on one variant. Each change was applied to both routes as a real edit. Both edited routes then had to release the same records to every requester on a fixture with records in a cancer gene, in a cardiac gene, in neither, and at HFE C282Y.

The fixed ACMG SF v3.2 gene set is used as a recognizable research query domain. Retrieving a ClinVar classification in that set does not apply all inheritance-, gene- and variant-specific criteria for secondary-findings reporting [38]. The demonstration also does not validate consent, authenticate requesters or prevent access outside the release evaluator. Those are separate clinical and deployment requirements.

S5.3 Outputs, stage costs and applied changes

Linking results. The SPDI linker emitted links for eligible explicit alleles under the case’s reference and normalization assumptions (Table S9). A seven-variant ClinVar *BRCA1* check found agreement with NCBI’s expressions outside repeats, while repeat-region expressions differed because minimal trimming and NCBI’s contextual representation use different spans. This is not a demonstration of canonical identity. The live tier confirmed 9,223 of 9,951 and 9,479 of 10,316

PGP rsID links in 21 MyVariant.info requests. All three unconfirmed rsIDs inspected had merged identifiers; the other unconfirmed cases were not established by those checks. The tier-3 Ensembl variation linker met a persistent HTTP 500 from Ensembl’s REST service after three retries and published no partial linkset.

Table S9: Linking on the use case’s real inputs: calls linked, summed over the arm’s files. SPDI eligibility in this implementation requires explicit alleles. Symbolic alleles are excluded from that linker. Arm 1’s gene count exceeds its SPDI count by 19 HG002 calls that the SPDI linker skips as ineligible but that still overlap a gene.

Linker (tier)	Arm 1	Arm 2	Arm 3	ClinVar	Result
spdi (2)	51,302	1,149,198	3,887,200	331,413	Every eligible call; joins genomes to ClinVar across contig names; 87s for the whole genome
ensembl-genes-grch38 (2)	51,321	1,149,198	1,704,921	—	Each file reloads the 108 MB GFF3, 19 of arm 2’s 22 min of linking
rsid-dbsnp (1)	20,267	—	—	—	The PGP genomes’ rsIDs, rewritten unchecked
rsid-myvariant (3)	18,702	—	—	—	92–93% of the tier-1 links confirmed in 21 requests; the three unconfirmed rsIDs checked had been merged
rsid-ensembl (3)	0	—	—	—	A persistent HTTP 500; failed closed, no partial linkset

Answers. The two routes returned identical requester-specific match sets and all release checks passed. P/LP-classified matches numbered zero in arms 1 and 3 and eight in arm 2. These are not clinically adjudicated reportable findings. Whole HG005 returned the same 1,496 tuples as its restricted arm-1 input. The cohort’s additional AF query joins a separate 68,635-site graph (6.5M triples) through SPDI identifiers; it replaces approximately 857M triples of repeated panel-level INFO by factoring information once, not by compressing equivalent graph structure. The baseline’s position-and-allele lookup agrees for every requester.

Table S10: Participant–variant–gene matches each requester may receive, identical between the RDF route and the bcftools route in every cell: all (no policy), clinical care (CC), the cardiovascular disease-specific study (DS) and the biobank’s general research use (GRU). Triples are the genomes’ graphs, before links. *Rare*: matches to variants with panel AF < 0.01, a join across the genome, ClinVar and the panel’s frequency file.

Arm	Files	Records	Triples	All	CC	DS	GRU
1: five genomes	5	51,386	9.0M	7,211	2,878	2,988	1,987
2: cohort	104	1,150,288	67.8M	173,102	63,529	92,870	48,207
rare in the panel				4,154	1,621	2,198	1,144
3: whole genome	1	3,887,810	662.4M	1,496	1,496	0	0

Cost. Query times differ much less than graph sizes in this case (Table S11): a cold-cache match query took 172–181s in arm 1, 176–202s in arm 2 and 205–209s over arm 3’s 701M-triple index, consistent with a dominant contribution from the fixed ClinVar join, rather than evidence of generally scale-independent querying. Building the index is what scales, from about 2 minutes to 45. Governance was first bound by Python rather than QLever: in arm 2 each view took 20–27 min to write, because every line was checked against each of 109 rules. Merging the binding rules’ selections once and filtering the inputs in parallel cut that to 131–165s, and resolving each term’s ancestors once cut it to 59–80s, with byte-identical views. Arm 3’s clinical view, every record released, streamed in 38 min on two workers, one per input file. Arm 3 peaked at 13.1 GB of memory and 34 GB of disk.

Effort. The RDF route’s rules are 72 lines, 41 of policy and 31 of query, against 131 lines of script; its linkers, selectors and profile ship with the tool (Table S12). Three of the four changes are data edits of similar size in both routes, because the baseline was written to read its rules from a table. They differ where a change introduces a new kind of rule. Withholding one named variant except for clinical care needs three new lines of code in the baseline, and six lines of policy without modifying the RDF policy engine. Stated on the variant’s SPDI identity, the rule also

Table S11: Cost of the RDF route per arm, on 8-core, 33.6 GB machines. Triples are those in the unrestricted query’s index: the genomes, ClinVar and all links. Convert and link are totals over the arm’s files. View and check are per requester: writing the view, then checking it, which for a non-empty view includes building its index. Index build and query are per requester too: three cold runs of the match query over that requester’s view, ClinVar and the links, the cache cleared before each, after the index is built.

Arm	Triples	Convert	Link	View	Check	Index build	Query
1	42.5M	3.3 min	1.4 min	32–39 s	21–37 s	2.2 min	172–181 s
2	110.3M	31 min	22 min	59–80 s	3–5 min	2.4–3.3 min	176–202 s
3 (clinical)	701.5M	54 min	8.0 min	38 min	68 min	45 min	205–209 s
3 (empty views)				28 min	1 min	2.2 min	<1 s

holds for HG002, whose file writes the contig as 6; a selector matching the contig string would have silently missed it. All four edited routes still agreed on the fixture.

Table S12: What each route’s author writes (arm 1) and the lines four changes add and remove. Blank lines, comments and docstrings are not counted. After each change both routes were re-run on a fixture and released the same records to every requester. POA is DUO’s population-origins-or-ancestry purpose; it sits outside general research use, so a GRU consent does not cover it.

	RDF route	bcftools route
Rules written	72: policy 41, query 31	131: shell 33, Python 98
Data written	28 gene IRIs; the query’s gene list (1 line)	consent and requester tables (100 lines); gene list (81)
Linkers, selectors, profile	shipped with the tool	—
A participant withdraws	policy +2/ – 1	data +2/ – 1
A requester with a new purpose (POA)	purpose vocabulary +2	data +4
The panel becomes 40 cardiac genes	policy +40/ – 28 (gene IRIs)	data +41/ – 29 (gene symbols)
One variant (HFE C282Y) clinical care only	policy +6, no engine change	code +3, data +9/ – 1

The whole-genome arm permits every genomic record for clinical care and none for the other requesters. It does not establish selective, non-empty release at whole-genome scale or masking within one multi-sample VCF. Exact lines of code are descriptive artifacts of these implementations; no participants were observed using the software, and no usability or maintenance advantage is measured.

S6 Conversion, storage and retrieval measurements

S6.1 Record and sample ladders; artifact accounting

Records. On the HG005_GRCh38 ladder (Figure S5a), triple production is approximately proportional to records (fitted exponent +0.999, 95% CI [+0.991, +1.007]), at a constant 170.9–171.9 triples per record. Final artifact bytes scale at +1.033, peak workspace at +1.025 and end-to-end wall time at +1.063 (40.7 s, 423.7 s and 5,476.0 s at 10 k, 100 k and 1 M records; replicate $sd \leq 13.5$ s). Peak resident memory of the mapping stage stays between 1.0 and 1.7 GB across the hundredfold change: the measured mapping stage is memory-bounded over this ladder; this does not establish bounded memory for representation builders, validators or query engines. The RMLStreamer mapping stage accounts for only 12.2–41.4 s of those runs; building the queryable representations dominates.

Samples. Re-emitting the same 10,000 records against 1 to 2,504 samples separates the profiles as Equation 1 predicts (Figure 2). Both series are exactly linear: $1,359,498 + 250,005 S$

triples expanded (25 per sample per record, plus 5 per sample of declarations; the fitted exponent of +0.795 reflects the fixed record layer at low S) and $1,439,498 + 5S$ condensed, so at 2,504 samples the expanded graph has 627,372,018 triples, $432\times$ the condensed one. Triple counts mislead about size. Condensed values move into vector literals, so the condensed HDT grows $5.8\times$ (10.2 to 59.2MB) while its triples grow 0.9%, and the byte gap at 2,504 samples is $322\times$ in gzip-framed N-Triples and $69\times$ in HDT. The condensed HDT is smaller than the uncompressed VCF only from 256 samples (10.2 against 1.5 MB at one sample; 59.2 against 101.6 MB at 2,504); the expanded HDT is $40\times$ the VCF (4.10 GB) and took 15,269s to build end to end, against 250s condensed. Extrapolated to the full 1,812,841-record call set, the expanded profile would need roughly 2 TB against a 189 GB volume, so the tool refuses such a run and names the condensed profile.

Storage mode. Plain and space-optimized storage give identical triples and the same final artifacts, to within one 4kB filesystem block, and differ only in transient disk (Figure S5b). Space-optimized mode cuts peak host workspace by $9.19\times$ (2.98 GB to 0.325 GB) on the 100k-record input and by $7.68\times$ (43.40 GB to 5.65 GB) on **test-larger**, a 1,155,741-record, single-sample prefix of the NG131FQA1I genome, at a time cost of +5.2% and +3.2%. On the replicated input the mapping stage is equivalent within a $\pm 10\%$ margin fixed before the run (+0.0%, 90% bootstrap CI $[-1.9\%, +0.4\%]$).

Representations. Across the corpus (Figure S4), COTTAS is consistently the smallest artifact: $0.37\text{--}0.55\times$ the stored gzip-framed N-Triples and $0.27\text{--}0.37\times$ the HDT built from the same graph. Uncompressed HDT is *larger* than the gzip-framed N-Triples it was built from, by $1.36\text{--}1.82\times$. COTTAS takes $1.5\text{--}3.0\times$ longer to build than HDT (31,230s against 20,911s on the whole HG005_GRCh38). Peak workspace is $4.7\text{--}30.3\times$ the uncompressed input. The whole genome, 657.4M triples, took 16.07h end to end, of which the mapping stage was 146.8s and the two representation builds 14.5h; the retained alternatives and HDT index (HDT 4.63 GB, its index 4.74 GB, COTTAS 1.50 GB) occupy a measured 10.88 GB output tree, $78\times$ the 139 MB compressed download, not a minimal required RDF footprint, and the gzip-framed N-Triples alone 2.89 GB.

S6.2 Costs with different meanings

The 16.07-hour v3.1.0 corpus workflow built both HDT and COTTAS from the 657.4M-triple graph. Its 146.8-second RML mapping stage is not total VCF-to-RDF conversion: emitters and other stages are outside that number. The 53-minute v3.2.0 conversion of the normalized HG005 genome (54 min with ClinVar) produced 662.4M genomic triples and is followed by separate linking, release, checking and indexing stages. The two timings differ in version, inputs and work performed; subtracting them does not estimate a speed-up.

A gzip-framed N-Triples archive retains the RDF stream (2.89 GB for the corpus whole genome). A QLever workflow additionally needs its own built index and loading workspace. An HDT or COTTAS archive is an alternative representation with separate construction and native access paths. Retaining both plus an HDT index explains the reported 10.88 GB total; it is not the size required for every deployment. The 10.88 GB is the measured output tree (10,879,410,176 bytes); the rounded components sum to 10.87 GB.

S6.3 Retrieval paths and conventional baselines

Engine and physical input path. On the 17.1M-triple graph, QLever’s thirteen-question totals from N-Triples, HDT and COTTAS differ by under 1% (17.0–17.1 s), and no single question by more than 0.07 s (Figure 3c). On the 0.96M-triple graph the thirteen questions cost a mean of 1.1 s under QLever, 23.4 s under Comunica, 38.2 s under the COTTAS engine and 45.1 s under the HDT-backed engine (Figure 3b), all four agreeing in every run.

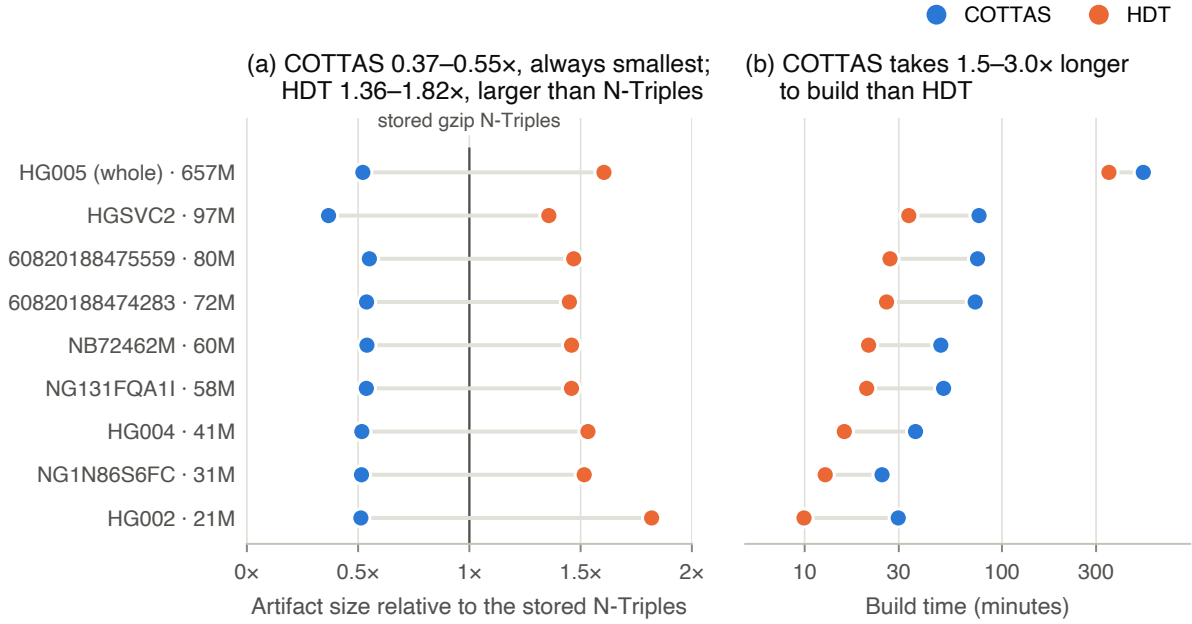


Figure S4: **Queryable representations in the corpus experiment**, one run per completed input, ordered by graph size (triples after each label). Eight files use their first 250,000 records with headers retained; HG005_GRCh38 is processed whole. The 2,504-sample 1000 Genomes cell is guard-skipped in expanded mode. Dataset labels are abbreviated; full names are in Table S3. (a) Artifact size relative to the stored gzip-framed N-Triples that space-optimized mode writes. (b) Build time of each representation from the same graph, on a log scale. Every artifact passed the tool’s native streamed decode and source-versus-decoded triple-count check.

Against a VCF parser, the answer depends on the workload. For a *single* question on the 17.1M-triple graph, cyvcf2 must scan the VCF whichever question it is, at 11.5s (12.3s for the three sample-level questions), while QLever answers in 0.005–4.62s once its index is built: 2.5× to 2300× faster (Figure 3a). That index takes 22.8s, so the parser is faster for the tested individual cold-start questions. For the *whole* thirteen-question batch, cyvcf2 answers everything in one 12.4s pass, against 17.1s of QLever query time plus the 22.8s index.

Large-graph retrieval in the tested workload. The thirteen questions cost 1.13, 1.00, 0.96 and 0.97s per million triples at 0.96M, 17.1M, 171M and 657M, and the index 1.27–1.35s per million above the smallest graph (Table S13; the two larger graphs were queried on a third host of the same specification). On the whole genome they took 634.6s after an 868.0s index build, replicates within 0.7%, every answer equal to the parser’s. With QLever the input artifact has little effect on the query phase (634.6, 636.3 and 636.6s from N-Triples, HDT and COTTAS); only the one-time decode does (179–192, 955–972 and 592–601s). Neither Comunica-backed engine completes the workload at 171M triples, even with a 24 GB heap: each answers five questions, the HDT engine its fifth only at the one-hour per-query ceiling, and then both fail on the sixth, the COTTAS engine unable to allocate. On those five the COTTAS engine is 16.7× faster, where at 0.96M the two were within 1.2×, so engine orderings at fixture scale do not predict behavior on a cohort.

Against indexed VCF access, the two cross over. Across 11,160 regional executions the arms recorded 0 failures and 0 disagreements (Table 5). On the 17.1M-triple graph QLever’s cost is almost independent of window size, 9.6ms at 1kb and 11.1ms at 10Mb, under this workload and query plan. The indexed readers’ cost tracks the records a window contains, growing 35.7× for cyvcf2 and 7.5× for bcftools over the same range. At 1kb they win, cyvcf2 by 2.9× and bcftools by 2.2×; by 1Mb QLever has overtaken cyvcf2, and at 10Mb it leads both, by 10.6× and 2.9×.

QLever’s index took 23.2s against 0.48s to bgzip and tabix-index the file, and the in-memory engines cost 896–1005 ms per 1 kb window on the smaller graph against QLever’s 18.8 ms, so they were not carried further.

S6.4 Setup amortization is workload-specific

For a repeated single-question workload with a positive per-question saving,

$$N_{\text{break-even}} = \frac{\text{additional RDF setup}}{\text{VCF time per question} - \text{indexed RDF time per question}}. \quad (2)$$

The reported 100,000-record scenario gives $(423.7 + 22.8)/(11.5 - 1.32) \simeq 44$ questions. The 423.7 seconds includes HDT construction, so this is not the minimal N-Triples-plus-QLever setup. It assumes repeated scans at the measured question mix, not a single parser pass computing all answers. A single batched parser pass over the thirteen questions took 12.4s versus 17.1s for the indexed SPARQL queries before setup, changing the conclusion. No universal break-even count follows.

S6.5 Record-scaling figure and complete large-graph outcomes

Figure S5 and Table S13 hold measurements that Section 3.3 of the main text reports in the text.

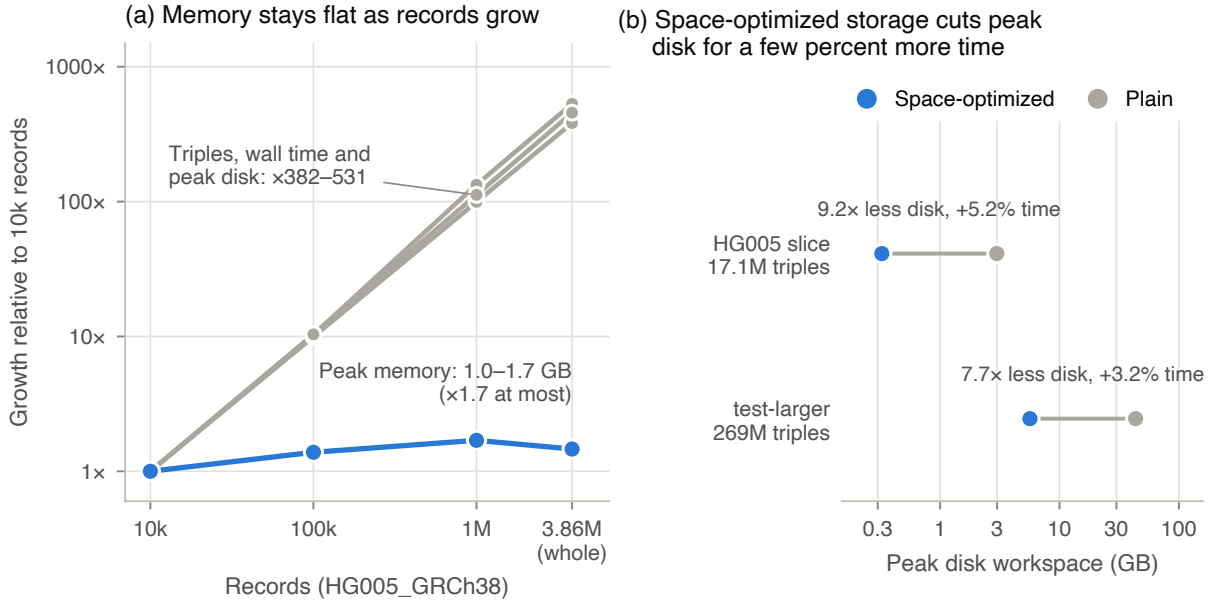


Figure S5: **Record scaling and storage mode.** (a) Growth of each measure relative to the 10,000-record rung of the HG005_GRCh38 ladder (medians of three replicates; the whole file is one run). Triples, end-to-end wall time and peak disk workspace grow in proportion to the records, while the mapping stage’s peak resident memory stays between 1.0 and 1.7GB. (b) Peak disk workspace of the plain and space-optimized storage modes on two inputs, with the end-to-end time cost of space-optimized mode. Final artifacts and triple counts are identical in both modes. Both panels are from the v3.1.0 run on `vcf-bench-1`.

Table S13: Retrieval at scale, on graphs built from HG005_GRCh38 by the published v3.1.0 image. The two smaller graphs were queried on **vcf-bench-1**, the two larger on **vcf-bench-3** (Section S1), so the per-million figures are descriptive and do not remove host variation. Times are for the thirteen core questions of Table S7, excluding the one-time index build, which is reported separately. Every completed question’s answer was compared against cyvcf2 before its timing was read. The Comunica-backed engines do not complete the workload at this size; their totals cover the five questions they answered, and are not comparable with a thirteen-question total. The three input packagings differ by 0.3% in query time after QLever indexing; their one-time decode costs differ.

Graph	Triples	Engine	Time (s)	Outcome
<i>Does SPARQL scale to a whole genome? (all thirteen questions)</i>				
Fixture	958,919	QLever	1.1	1.13 s per million triples
100 k records	17,098,746	QLever	17.0	1.00 s per million
1 M records	170,935,101	QLever	163.3	0.96 s per million
Whole genome	657,425,805	QLever	634.6	0.97 s per million; setup 868.0 s; replicates within 0.7%
<i>Does the artifact matter? (whole genome, one engine, three packagings)</i>				
from N-Triples	gzip-framed		634.6	Decode 179–192 s; setup 868.0 s
from HDT	—		636.3	Decode 955–972 s; setup 861.8 s
from COTTAS	—		636.6	Decode 592–601 s; setup 868.4 s
<i>Do the other engines get there at all? (1 M records, 170,935,101 triples)</i>				
COTTAS engine	5 of 13 answered		743.1	Native allocation failure on Q6
HDT engine	5 of 13 answered		12,432.2	Q5 answered at the one-hour per-query ceiling (3,600.1 s); Q6 failed

S7 Configuration coverage, operating modes and memory feasibility

Covering set. A ten-row covering set exercised every value of all seven configuration axes — sample representation, INFO representation, storage mode, RDF compression, representations, artifact compression and HDT strategy — and all ten rows exited 0 and passed validation, including the shapes. Pairwise coverage is complete except for three combinations the wrapper structurally refuses (**-hdt-strategy single** beside space-optimized storage, beside COTTAS, and beside **hdt,cottas**), so the remaining gaps are specification, not omission. Gzip and Brotli were exercised as values of both compression axes; the campaign does not compare the two codecs’ ratios or throughput.

Operating modes. Each of the tool’s operating modes — **tsv**, **compress**, **decompress**, **index** for HDT and for COTTAS, a conversion-only phase and a validation-only phase — ran independently and exited 0, confirming that conversion and validation are separable stages rather than one monolithic command.

Memory feasibility. A feasibility sweep crossed three memory ceilings (8, 16 and 31 GB) with three configurations on a one-million-record input. All nine cells completed; peak resident memory was 1.34–1.93 GB in every case, below every requested ceiling, which is consistent with the flat memory scaling in Section 3.3 of the main text. This experiment carries one caveat that must be reported with it: the ceiling was requested through Docker memory limits but the harness could not confirm that the pinned release forwards them to its container invocations, so these cells establish that the workload is memory-light, not that it survives a genuinely enforced cap.

The cost of the INFO and header representations. On the structural-variant file, structured INFO produced 96.75M triples against 89.01M for raw INFO (+8.7%), a 7.4% larger N-Triples output and a 7.8% larger HDT, and on the 100,000-record HG005_GRCh38 input it produced 17.10M against 12.52M (+36.6%). Raw INFO still carries the allele layer here, because the expanded sample profile references it (Section S8); before that fix the raw arm omitted it and the difference looked larger than it is. Structured header representation costs almost nothing by comparison (17,098,746 against 17,094,293 triples, +0.03%).

Cross-encoding equivalence. The equivalence experiment converted one 10,000-line (9,986-record), single-sample fixture in both sample representations and both storage modes, built HDT and COTTAS from each, and ran the full twenty-seven-query suite (the fourteen preflight checks and Q1–Q13) under all four engines against all three artifacts. All sixteen validations passed with all four engines in agreement, on graphs of 0.96M triples (expanded) and 0.79M triples (condensed). The complete condensed cell took 70.4 min end to end. Two further cells asserted that the tool must refuse `-hdt-strategy single` beside COTTAS and beside space-optimized storage, and it did, with exit status 2.

S8 Defects, corrections and remaining gaps

Testing identified problems in measurement, conversion, and validation. Three of them were defects that the record counts alone could not show: two conversion defects that the validation layer caught, and one policy-plug-in defect that the plug-in’s own check caught; the main text summarizes them in Section 3.1 of the main text. The corrected outputs are reflected in the reported counts. Open oracle/normalization failures and archived unsuccessful runs remain part of the evidence, rather than being relabeled as successful validation.

Header-only input omitted its declared samples. A VCF with a declared sample column but no data records produced a graph without the `vcfc:SampleSet` and `vcfc:VCFSample` resources declared by its header. The emitters derived the file IRI from a source name learned from the first data row. The predicate and class censuses (Q9 and Q10) detected the missing resources. Emitting the sample set from the header increased the graph from 181 to 189 triples and allowed validation to complete. The same test exposed an incorrect oracle expectation: `preflight_sample_gt_inventory` expected one `vcfc:sampleId`, although that property belongs to a per-record `vcfc:SampleCall`. The oracle was adjusted to expect no such calls when the file contains no records.

Allele references without corresponding resources. Applying the shapes to the configuration covering set identified three failing rows, all combining raw INFO with expanded samples. Allele resources were emitted only for structured INFO, but expanded genotypes referenced them through `vcfc:calledAllele` regardless of the INFO setting. A 1,000-record test produced 1,972 such references without the corresponding allele descriptions. The comparison queries counted those edges without following them, whereas the shapes reported 5,916 violations. The emitter was changed to produce the allele layer when either representation requires it. The census oracle also needed the same dependency rule: its initial expectation reproduced the emitter’s assumption and rejected the added triples. After both were updated, all ten covering-set rows passed. Each affected row gained 10,846 triples, while the other seven outputs remained byte-identical. Regression checks cover all four INFO/sample representation combinations and require every referenced allele to be described.

A withdrawn file’s header remained in every release view. During development of the policy plug-in, every record decision was correct, but a withdrawn file’s header remained in every view, which the record counts alone could not show. The plug-in’s own check caught it. That defect is why the plug-in defines what a withheld resource takes with it by an ownership rule rather than by a list of record subtrees (Section 2.2 of the main text).

Two memory guards that measured the wrong quantity. Extending retrieval to 171M triples (Section 3.3 of the main text) exposed two defects, neither visible at fixture scale. First, the shape layer is size-gated because `pyshacl` is in-memory, but the gate weighed the *packaged artifact* while the cost it guards against is set by the *graph*. One 170,935,101-triple graph therefore fell on both sides of a single 512 MiB threshold according only to how it was packaged: as COTTAS (390.7 MB) it passed the gate and the run was killed at 30.7 GiB resident on a 31.3 GiB host, while the same graph as gzip-framed N-Triples (756.6 MB) or HDT (1.18 GB) was skipped. The better

a format compressed, the likelier it was to exhaust memory. The gate now counts triples, which no packaging changes. In `-mode full` the same release’s gate weighs the input VCF rather than the graph, so it admitted a 58.2M-triple real-genome graph, from a 63.8MB input, to `pyshacl` during this study’s validation of `NG131FQA1I` (Section 3.1 of the main text); the host ran out of memory, and that validation was rerun without shapes. Second, the Comunica-backed endpoints aborted with a JavaScript heap exhaustion while roughly 25 GB of the host’s memory was free, because Node does not size its heap from the machine and no ceiling was set; the ceiling is now configurable. Both corrections are released in v3.3.0 and neither alters conversion output; the failing runs are archived alongside the successful ones.

COTTAS query execution and timeout enforcement. The initial COTTAS query implementation used `pycottas`’ `RDFLib` store. On the condensed representation it completed the fourteen preflight queries and Q1–Q4, then failed to complete `q05_sample_genotype_counts`. Two attempts ran for approximately 41 h and 10 h at roughly 190% CPU before being stopped. The second had a 1,800s per-query timeout and a 14,400s validation budget, but neither interrupted the in-process query. Both runs are archived. Replacing this path with the DuckDB-backed Comunica engine reduced Q5 to 2–5s and the thirteen-query workload on the 0.96M-triple comparison graph to 38s, against 1,402s through `RDFLib`. Query execution now occurs where the timeout can terminate it, and the completed cross-engine measurements use this engine.

SHACL report interpretation. Integrating the vocabulary’s SHACL shapes exposed two reporting errors. The validator treated `pyshacl`’s `conforms` flag as a blocking verdict even for recommendations, causing conformant INFO declarations without `Source` to be rejected. Its violation matcher also expected a report format that `pyshacl` did not emit. The parser was corrected to classify results by severity, with only `sh:Violation` blocking a run.

Missing-token preflight false positives. The preflight reports contain 608 checks: 599 passed and nine flagged the missing-token rule, all on the 100,000-record `HG005_GRCh38` input. The rule looks for bare `"."` literals that should have been typed as `vcfc:Null`. It first returned 20 rows there, all on `vcfc:fieldNumber` or `vcfc:genotypeString`, where a dot is legitimate: `Number=.` denotes variable cardinality, and a fully missing genotype is written `..`. The rule was narrowed to exclude those two predicates. In v3.1.0 it still returns ten rows, and they are the same case one layer over: the header declares ten INFO fields with `Number=.`, and the header-attribute layer also carries each declaration’s `Number` attribute verbatim as a `vcfc:attributeValue` of `"."`. A follow-up change excludes that attribute’s value as well while still reporting a bare dot on any other attribute; it postdates v3.1.0, so these nine flags (ten rows each) remain in this campaign’s reports. None of the flags reflects malformed output.

Sites-only fixture limitation. The sites-only fixture produced 245 triples and a valid HDT artifact, but its two validations ended with `EXECUTION_FAILED`. The fixture declared a `FORMAT` column without sample columns, which `bcftools` rejects. This is a fixture/oracle limitation rather than evidence that the generated graph failed semantic comparison. No completed paired result is claimed for this input, and a conformant sites-only fixture remains necessary to establish that coverage.

Measurement and reporting in the legacy benchmark. The earlier six-file, batch-oriented benchmark exposed two measurement problems. Partitioned compression wall times were under-reported; the timing implementation was corrected in commit `fb48e50`, and the affected historical durations were reconstructed from wrapper logs and execution timestamps. In addition, an input-size column labeled `.vcf.gz` actually contained the uncompressed VCF stream size: the retained `NG131FQA1I.vcf` contains 1,215,509,730 bytes, matching the reported 1215.5 MB. The compression denominator also affected interpretation: the legacy HDT reduction of approximately 92–95% was measured against uncompressed N-Triples, whereas in the current corpus experiment HDT is 1.36–1.82 times the size of the stored gzip-framed N-Triples. These use different reference artifacts and are not a like-for-like change in compression performance, so the legacy conversion and compression tables are excluded from the current analysis.

Review-run oracle failures remain open. The 58.2M-triple NG131FQA1I follow-up exposes two classes of validator defect: omitted phase-set expectations and a lexical decimal dependency in the record digest (Section S4). Diagnostic recomputation explains the observed failures, but a corrected validator has not yet been released. Extending that oracle to structural-variant events, gVCF blocks and local-allele constructs is also outside the completed evidence. These issues are distinct from the fixed missing-sample, allele-resource and ownership defects.

References

- [1] Rafael C. Jiménez, Mateusz Kuzak, Monther Alhamdoosh, Michelle Barker, Bérénice Batut, Marc Borg, Salvador Capella-Gutiérrez, Neil P. Chue Hong, Martin Cook, Manuel Corpas, Michael Flannery, Leyla Garcia, Josep L. Gelpí, Simon Gladman, Carole Goble, Miguel González Ferreira, Alejandra Gonzalez-Beltran, Philip C. Griffin, Björn Grüning, Jon Hagberg, Petr Holub, Rob Hooft, Jon Ison, Sarah Jones, Florian Kaiser, Sunghwan Kim, Matthew Krafczyk, Anna Krystalli, Hilmar Lapp, Ann Lonie, Céline Marek, Michael P. Mazanetz, Aleksandra Nenadic, Wolfgang Petersen, Esther Plomp, Mikhail Pomazkov, Karthik Ram, Stian Soiland-Reyes, Alexey Sokolov, Andreas Struck, Radoslaw Suchacki, Rutger Vos, Alan R. Williams, Patrick Brack, Marta W. Muelas, Alexander D. Diehl, Jean-Baptiste Poline, Teresa K. Atwood, Alexander Mansurov, Larisa N. Soldatova, Carl Kingsford, Paul J. Kersey, Mark D. McDowall, Magne Østeras, Cristina Guijarro-Clarke, and Steve Crouch. Four simple recommendations to encourage best practices in research software. *F1000Research*, 6:876, 2017.
- [2] Elias Crum. VCF Core Vocabulary. Version 2.1.3. GitHub repository, 2026. Version 2.1.3 with the SSSOM alignment set; source snapshot of 22 September 2026.
- [3] Jerven T. Bolleman, Christopher J. Mungall, Francesco Strozzi, Joachim Baran, Michel Dumontier, Raoul J. P. Bonnal, Robert Buels, Robert Hoehndorf, Takatomo Fujisawa, Toshiaki Katayama, and Peter J. A. Cock. FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. *Journal of Biomedical Semantics*, 7(1):39, December 2016.
- [4] Karen Eilbeck, Suzanna E Lewis, Christopher J Mungall, Mark Yandell, Lincoln Stein, Richard Durbin, and Michael Ashburner. The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology*, 6(5):R44, April 2005.
- [5] Laura Menotti, Mirco Cazzaro, Manuel Rueda, Ivo G Gut, and Gianmaria Silvello. HERO-Genomics: Bridging Genomic Data and Ontological Modelling. In *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, Barcelona, Spain, February 2025. CEUR Workshop Proceedings.
- [6] Alex H. Wagner, Reece Hart, Larry Babb, Kori Kuzma, Helen Schuilenburg, James Stevenson, Terry O'Neill, Mrinal-Thomas Epic, Brad Holmes, Eric Moyer, Michael Baudis, Peter Robinson, Melissa Cline, Ronak Y. Patel, and Robert R. Freimuth. ga4gh/vrs 2.0.1, 2025.
- [7] Arto Bendiken, Raoul J.P. Bonnal, and Francesco Strozzi. ruby-rdf/rdf-vcf: Rdf vcf parser, November 2025.
- [8] Emanuel Diego S. Penha, Egiebade Iriabho, Alex Dussaq, Diana Magalhães de Oliveira, and Jonas S. Almeida. Isomorphic semantic mapping of variant call format (VCF2RDF). *Bioinformatics*, 33(4):547–548, 2017.
- [9] Toshiaki Katayama, Shuichi Kawashima, Gos Micklem, Shin Kawano, Jin-Dong Kim, Simon Kocbek, Shinobu Okamoto, Yue Wang, Hongyan Wu, Atsuko Yamaguchi, et al. Biohackathon

series in 2013 and 2014: improvements of semantic interoperability in life science data and services. *F1000Research*, 8:1677, 2019.

- [10] Pierre Lindenbaum. JVarKit: java-based utilities for Bioinformatics. figshare, 5 2015.
- [11] Nobutaka Mitsuhashi, Licht Toyo-oka, Toshiaki Katayama, Minae Kawashima, Shuichi Kawashima, Kazunori Miyazaki, and Toshihisa Takagi. TogoVar: A comprehensive Japanese genetic variation database. *Human Genome Variation*, 9(1):44, December 2022.
- [12] Jasmin K. Böhmer and Roel Janssen. Linked data in practice: An RDF-based approach with SPARQLing Genomics, June 2020. Version V4.
- [13] Roel Janssen. SPARQLing Genomics. Open-source software, 2019. Accessed 2026-09-03.
- [14] Joachim Baran, Bibi Sehnaaz Begum Durgahee, Karen Eilbeck, Erick Antezana, Robert Hoehndorf, and Michel Dumontier. GFVO: the Genomic Feature and Variation Ontology. *PeerJ*, 3:e933, 2015.
- [15] Erik Garrison, Zev N. Kronenberg, Eric T. Dawson, Brent S. Pedersen, and Pjotr Prins. A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar. *PLOS Computational Biology*, 18(5):e1009123, 2022.
- [16] Shivika Prasanna, Ajay Kumar, Deepthi Rao, Eduardo J. Simoes, and Praveen Rao. A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning. *Frontiers in Big Data*, 7:1466391, 2025.
- [17] Pierre Larmande, Bertrand Pittolat, Ndomassi Tando, Yann Pomie, Bill Gates Happi Happi, Valentin Guignon, and Manuel Ruiz. AgroLD: a knowledge graph for the plant sciences. *BMC Genomic Data*, 26(Suppl 1):73, 2025.
- [18] Alexandrina Bodrug-Schepers, Hugo Chabane, Gabriela Montoya, Richard Redon, Alban Gaignard, and Patricia Serrano-Alvarado. Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. In *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, Barcelona, Spain, February 2025. CEUR Workshop Proceedings.
- [19] Ben De Meester, Pieter Heyvaert, and Thomas Delva. RDF Mapping Language (RML), June 2024.
- [20] Sitt Min Oo, Gerald Haesendonck, Ben De Meester, and Anastasia Dimou. RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In Ulrike Sattler, Aidan Hogan, Maria Keet, Valentina Presutti, João Paulo A. Almeida, Hideaki Takeda, Pierre Monnin, Giuseppe Pirrò, and Claudia d’Amato, editors, *The Semantic Web – ISWC 2022*, volume 13489, pages 697–713. Springer International Publishing, Cham, 2022. Series Title: Lecture Notes in Computer Science.
- [21] RDF/HDT contributors. hdt-cpp: HDT C++ library and tools, 2026. Built from the repository head at image build time; the exact revision is fixed by the VCF-RDFizer v3.1.0 image digest.
- [22] Javier D. Fernández, Miguel A. Martínez-Prieto, Claudio Gutiérrez, Axel Polleres, and Mario Arias. Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web*, 19:22–41, 2013.
- [23] FRINK OKN. hdtc: HDT creator, 2026. Version 1.1.0, computer software, MIT License.

- [24] Dennis Diefenbach and José M. Giménez-García. HDTCat: Let’s make HDT generation scale. In *The Semantic Web – ISWC 2020*, volume 12507 of *Lecture Notes in Computer Science*, pages 18–33, Cham, 2020. Springer.
- [25] Julián Arenas-Guerrero. pycottas: Python cottas library for compressing and querying rdf, 2026. RDFLib store backend, triple-pattern interface, and COTTAS conversion utilities.
- [26] Elias Crum. Comunica SPARQL COTTAS: a Comunica engine for query evaluation over COTTAS files, 2026. Version 0.1.0; npm package `@elias.crum/query-sparql-cottas`.
- [27] Petr Danecek, Adam Auton, Goncalo Abecasis, Cornelis A. Albers, Eric Banks, Mark A. DePristo, Robert E. Handsaker, Gerton Lunter, Gabor T. Marth, Stephen T. Sherry, Gilean McVean, Richard Durbin, and 1000 Genomes Project Analysis Group. The variant call format and VCFtools. *Bioinformatics*, 27(15):2156–2158, August 2011.
- [28] Petr Danecek, James K Bonfield, Jennifer Liddle, John Marshall, Valeriu Ohan, Martin O Pollard, Andrew Whitwham, Thomas Keane, Shane A McCarthy, Robert M Davies, and Heng Li. Twelve years of samtools and bcftools. *GigaScience*, 10(2):giab008, 2021.
- [29] Brent S. Pedersen and Aaron R. Quinlan. cyvcf2: fast, flexible variant analysis with Python. *Bioinformatics*, 33(12):1867–1869, 2017.
- [30] Hannah Bast and Björn Buchhold. QLever: A query engine for efficient SPARQL+Text search. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*, pages 647–656, 2017.
- [31] Ruben Taelman, Joachim Van Herwegen, Miel Vander Sande, and Ruben Verborgh. Comunica: A modular SPARQL query engine for the web. In *Proceedings of the 17th International Semantic Web Conference*, pages 239–255. Springer International Publishing, 10 2018.
- [32] Comunica Association. Querying local files from the command line, 2026.
- [33] Comunica Association. HDT: Querying HDT files with comunica, 2026.
- [34] J. Bradley Holmes, Eric Moyer, Lon Phan, Donna Maglott, and Brandi Kattman. SPDI: data model for variants and applications at NCBI. *Bioinformatics*, 36(6):1902–1907, 2020.
- [35] Jiwen Xin, Adam Mark, Cyrus Afrasiabi, Ginger Tsueng, et al. High-performance web services for querying gene and variant annotation. *Genome Biology*, 17:91, 2016.
- [36] Renato Iannella and Serena Villata. ODRL Information Model 2.2, February 2018.
- [37] Jonathan Lawson, Moran N. Cabili, Giselle Kerry, et al. The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics*, 1(2):100028, 2021.
- [38] David T. Miller, Kristy Lee, Noura S. Abul-Husn, Laura M. Amendola, et al. ACMG SF v3.2 list for reporting of secondary findings in clinical exome and genome sequencing: A policy statement of the American College of Medical Genetics and Genomics (ACMG). *Genetics in Medicine*, 25(8):100866, 2023.
- [39] Personal Genome Project, 2005.
- [40] Justin Wagner, Nathan D. Olson, Lindsay Harris, Ziad Khan, Jesse Farek, Medhat Mahmoud, Ana Stankovic, Vladimir Kovacevic, Byunggil Yoo, Neil Miller, et al. Benchmarking challenging small variants with linked and long reads. *Cell Genomics*, 2(5):100128, 2022.
- [41] Marta Byrska-Bishop, Uday S. Evani, Xuefang Zhao, et al. High-coverage whole-genome sequencing of the expanded 1000 Genomes Project cohort including 602 trios. *Cell*, 185(18):3426–3440.e19, 2022.

- [42] National Center for Biotechnology Information. ClinVar VCF Release (GRCh38), September 2026. Header declares `##fileformat=VCFv4.1`; accessed 8 September 2026.